

# Legion — hands on the twin

Opening, looking at, and running the digital twin on the Legion machine

## What this is

The three documents you want **at the keyboard on Legion**, in order. Goal: open the digital twin, look at it, click on it, press Play, then run the door task.

|   | Step                                       | Source                | Lines |
|---|--------------------------------------------|-----------------------|-------|
| 1 | The machine, its paths, and the kernel pin | environment_legion.md | 154   |
| 2 | Isaac Sim GUI — look at the twin           | ISAAC_SIM_UI_GUIDE.md | 190   |
| 3 | Docker + Isaac Lab — run the door task     | DIGITAL_TWIN_HOWTO.md | 235   |

## Start here, literally

```
cd /home/aidev/isaacsim
./isaac-sim.sh
```

~16 seconds to "app ready". Then **File → Open** and load `/home/aidev/RL_Twin/digital_twin_models-DT_reorg/RidgebackWithURGripper/RidgebackWithURGripper.usd` — one robot, not the full lab. Orbit with left-drag. Select something in the Stage tree and press **F** to fly the camera to it; that single key is the difference between exploring and being lost.

Then work down step 2's "Things to try" list, items 1 to 8. Item 5 opens `ENV_OT_UR_door/TrainingScene_flattened.usd`, which is the exact scene the RL training uses, and item 6 finds the prim that is the task:

```
/World/OpenTronsFlex/OpentronsDoorAdapter_large/LooRoll/Handle
```

Its `RevoluteJoint` is what "opening the door" physically means. Worth looking at, given the measurement that the gripper cannot get within 20 cm of it.

## Four things that will confuse you, all expected

- **A scary asset warning on every load** — Could not open asset `@file:/C:/Users/prat615/...OpentronsDoorAdapter_large.usdc@`, a stale path to Martin's own Windows machine. **The handle is genuinely there anyway**; the geometry was baked in when the scene was flattened. This caused a false alarm on Legion before anyone checked properly.

- **Not all actuators are configured! 7 != 15** — intended. Only some joints are motorised, and that is the same fact as the reachability problem: the base DOF are not among them.
- **Play vs Stop.** Not playing, you are looking at a static scene. Playing, physics is live. Always press Stop before editing, per Martin's own README.
- **Two Isaac Sims on the box, deliberately.** The workstation install at `/home/aidev/isaacsim` is for looking; the Docker container is for training. Both pinned to **5.1.0**, because Martin's assets were authored in 5.1 and the download page defaults to something newer.

## Before you touch a system update

---

The GPU works only on the pinned kernel `6.17.0-14-generic`, which is the one with a matching NVIDIA driver module. A routine Ubuntu upgrade that changes the boot kernel **takes the GPU out** — `nvidia-smi` starts failing and nothing Isaac-related runs. Check with `uname -r`. Step 1 explains the pin and how to revisit it.

## The one part that was never made to work

---

Rendering from *inside* the Docker container. Step 3 gives two candidate paths — `xhost +local:docker` and `--headless --livestream 2` with NVIDIA's WebRTC client — and marks both untested. If you want to watch a robot move, use the workstation GUI in step 2 instead; that route is verified.

Generated 2026-09-02 from `RL_DT@d827a53`, which is the commit that pulled the Legion work onto the PNNL machine. Assembled verbatim from the markdown sources named in each section header; nothing was rewritten for this packet, and the framing notes under each header are the only added text. Chromium print does not emit page numbers, so the section banners are the navigation.

---

## Step 1 of 3 — the machine, and what will bite you

RL\_DT/\_docs\_legion/environment\_legion.md - 155 lines

*Read the kernel-pin section before you touch a system update: the GPU driver module only exists for the pinned kernel, and a routine upgrade will take the GPU out. Also the disk layout, so the paths in steps 2 and 3 mean something.*

## Environment — trossen-ai (the Legion machine)

Companion to `environment.md`, which describes Dewei's laptop and the PNNL workstation. **That file is not modified**; this one records the machine the 2026-08-27/29 work ran on. Where the two disagree about paths or tooling, they are describing different computers.

### The machine

| Hostname | trossen-ai                                                  |
|----------|-------------------------------------------------------------|
| OS       | Ubuntu 24.04.2 LTS (Noble)                                  |
| Kernel   | 6.17.0-14-generic, pinned — see below                       |
| CPU      | Intel Core Ultra 9 275HX, 24 cores                          |
| RAM      | 64 GB                                                       |
| GPU      | NVIDIA GeForce RTX 5090 Laptop, 24 GB, Blackwell ( sm_120 ) |
| Driver   | 580.126.09, CUDA 13.0                                       |

### The kernel pin — do not undo this casually

`nvidia-smi` failed on arrival. The GPU was present and `nvidia-driver-580-open` was installed, but **no NVIDIA kernel module existed for the running kernel** ( `6.17.0-35-generic` ). Installed module packages covered only `6.17.0-14` and `6.14.0-37`.

Installing the matching module for `-35` is not currently possible: the archive's `linux-modules-nvidia-580-open-6.17.0-35-generic` depends on `nvidia-kernel-common-580` in a version range that contradicts what the archive actually ships ( `>= 580.159.03` and `<= 580.159.03-1`, while the archive has `580.173.02` ). Apt cannot resolve it.

**Resolution applied:** permanently default-boot the older kernel, which already has a working module.

```
sudo grub-set-default 'Advanced options for Ubuntu>Ubuntu, with Linux 6.17.0-14-generic'
```

This alone was **not** enough: `/etc/default/grub` had `GRUB_DEFAULT=0`, which hard-codes booting entry 0 and ignores the saved entry. It also needed:

```
sudo sed -i 's/^GRUB_DEFAULT=0/GRUB_DEFAULT=saved/' /etc/default/grub
sudo update-grub
```

**This is a "for now" fix, not permanent.** Periodically check whether Ubuntu ships a matching `580-open` module for a newer kernel and move back, so the machine keeps receiving kernel security updates. Check with `uname -r` and `dpkg -l | grep linux-modules-nvidia`.

## Network — what this machine can and cannot reach

On the `pnnl-devices` wifi. It has a PNNL-internal IP (130.20.215.70) and PNNL internal DNS, but is firewalled off from internal services — it is an unregistered-device network.

| Target                                                 | Reachable                                            |
|--------------------------------------------------------|------------------------------------------------------|
| Internet (github.com, pypi, nvcr.io, NVIDIA downloads) | yes                                                  |
| <code>tanuki.pnnl.gov</code> (PNNL GitLab)             | no — resolves to 172.26.60.16 but every port refuses |
| <code>\\pnl\users</code> , <code>\\pnl\projects</code> | no — does not resolve, SMB blocked                   |

**DNS resolution is not access.** `tanuki.pnnl.gov` resolving misled an early check here; a TCP connect to 443/80/22 all fail. Test connectivity, not name lookup.

Consequences: repos cannot be cloned or pulled from `tanuki`, so `digital_twin_models-DT_reorg` and `isaacclab_dt-main` exist here as **extracted copies, not git clones** — they cannot receive updates. The Isaac Lab image had to be built locally rather than pulled from the GitLab registry. Syncing to PNNL machines goes through a personal GitHub repo instead (see `WORKLOG_legion.md`).

## Disk layout

```
/home/aidev/RL_Twin/
├── RL_DT/                          the git repo (github.com/Dewei-Wang-xx/RL4DT-Toolkit)
│   ├── _docs/                     documentation, including these _legion files
│   ├── _patches/                  the TF/triton fix, standalone
│   └── _toolkits/
│       ├── toolkit-isaacclab/      copy for the Isaac Lab + Docker route
│       └── toolkit-isaacsim/       copy for the bare Isaac Sim route
├── SciOptControlToolkit/          the working toolkit clone (branch develop-dw)
├── digital_twin_models-DT_reorg/   Martin's USD assets (extracted, not a clone)
├── isaacclab_dt-main/             Alvika's Isaac Lab fork (extracted, not a clone)
└── _reference/                    papers, older copies (~4 GB)

/home/aidev/isaacsim/              Isaac Sim 5.1.0 workstation install (17 GB)
```

Disk: 1.9 TB total, ~1.3 TB free. Docker holds ~143 GB (of which ~62 GB is reclaimable build cache via `docker builder prune`).

## Python and the toolkit

`uv` is used throughout (installed to `~/local/bin/uv`, symlinked from a snap path).

```
cd <toolkit-copy>
uv venv --python 3.11 .venv
source .venv/bin/activate
uv pip install -r requirements.txt
uv pip install -e .
```

Resolved: Python 3.11.16, torch **2.13.0+cu130** (CUDA available, sees the RTX 5090), TF 2.21.0, gymnasium 1.3.0, gymnasium-robotics 1.4.2.

`gymnasium-robotics` is in neither `requirements.txt` nor `env.yaml` but is required by `envs/robotics_flat.py` and by `utests/test_registry.py::test_env`. Same category of gap as the `pysindy` / `scikit-learn` one noted in

environment.md.

`pip install -e .` needs the packaging fix (see `WORKLOG_legion.md`) — upstream `setup.py` omits every subpackage, so a strict editable install exposes only the top-level package.

## Isaac Sim — two installations, deliberately

---

### 1. Docker container (isaac-lab-base, 34.7 GB image, built from Alvika's fork)

```
cd /home/aidev/RL_Twin/isaacsim-dt-main/docker
docker compose --env-file .env.base -f docker-compose.yaml --profile base up -d isaac-lab-base
docker exec -it isaac-lab-base bash
```

`.env.base` was repointed at this machine: `EXTERNAL_USD_PATH` → `digital_twin_models-DT_reorg`, `EXTERNAL_PROJECT_PATH` → `SciOptControlToolkit`. Inside the container these appear as `envs_external/` and `external_projects/`. `aidev` was added to the `docker` group.

### 2. Workstation install (/home/aidev/isaacsim, 17 GB, Isaac Sim 5.1.0)

```
/home/aidev/isaacsim/isaac-sim.sh      # GUI
/home/aidev/isaacsim/python.sh <script> # scripting
```

Version 5.1.0 deliberately: NVIDIA's download page defaults to 6.0.1, and Alvika flagged selecting 5.1.0 to match Martin's assets. The container is pinned to 5.1.0 too, so both agree. Downloaded directly from `download.isaacsim.omniverse.nvidia.com` — no NGC login was needed.

Two packages had to be added to Isaac Sim's bundled Python for the toolkit to run there:

```
/home/aidev/isaacsim/python.sh -m pip install tensorboard gymnasium
```

Without `gymnasium` the agent reports *"Action space not valid for this agent"* — a misleading message, since the real cause is the import failing inside a `try/except`.

## Line endings

---

The toolkit was copied from Windows, so every file had CRLF. On Linux this makes `git status` report ~80 files changed when only a handful are. The copies under `_toolkits/` were normalised to LF at copy time; the working `SciOptControlToolkit` clone was **not**. When inspecting changes there, use:

```
git diff --ignore-cr-at-eol
```

and never `git add ..`

---

## Step 2 of 3 — look at the twin (start here)

RL\_DT/\_docs\_legion/ISAAC\_SIM\_UI\_GUIDE.md - 191 lines

*The workstation Isaac Sim install, no Docker and no RL. Its 'Things to try' list, items 1-8 in order, is the actual session. Two sections to read before you start: the warnings you can safely ignore, and what could not be verified.*

## Playing with the Digital Twin in the Isaac Sim GUI

A hands-on guide for exploring the digital twin visually — no Docker, no Isaac Lab, no RL. Just you, the simulator, and Martin's robot and lab models.

**Verified on trossen-ai, 2026-08-29.** Isaac Sim 5.1.0, RTX 5090. Where a step could not be verified without a person watching the screen, it says so explicitly.

---

### What this is

You have **two separate ways** to run Isaac Sim on this machine, and they do not interfere:

|                     | Where                | Best for                              |
|---------------------|----------------------|---------------------------------------|
| Docker container    | isaac-lab-base image | RL training (has Isaac Lab on top)    |
| Workstation install | /home/aidev/isaacsim | <b>Looking at things</b> — this guide |

This guide covers the second. It is the one to use when you want to see the twin, click on it, and understand how it is put together.

---

### Starting it

```
cd /home/aidev/isaacsim
./isaac-sim.sh
```

Takes ~16 seconds to reach "app ready". The first launch after install may take longer while shaders compile — this is normal, not a hang.

Verified: launches cleanly, no errors, RTX renderer active on the RTX 5090.

To stop: close the window, or Ctrl-C in the terminal.

If the window does not appear but the terminal shows startup messages, the display variable may be wrong. Try `DISPLAY=:1 ./isaac-sim.sh` — that is the display this machine uses.

---

### Opening the digital twin

**File → Open**, then navigate to:

```
/home/aidev/RL_Twin/digital_twin_models-DT_reorg/
```

Good places to start, easiest first:

| What                        | Path                                              |
|-----------------------------|---------------------------------------------------|
| A single robot (start here) | RidgebackWithURGripper/RidgebackWithURGripper.usd |
| Just the gripper            | Robotiq/2F140_reworked.usd                        |
| The door task scene         | ENV_OT_UR_door/TrainingScene_flattened.usd        |
| The full lab                | ENV_ESCBench/ESCBench.usd                         |
| Tube racking task           | ENV_TubeRacking_UR/TubeRacking.usd                |

The complete catalogue — every robot, scene, gripper and piece of labware — is in `digital_twin_models-DT_reorg/README.md`. Worth reading; it is Martin's own description of what each file is.

**Start with a single robot, not the full lab.** The lab scene has far more geometry and takes longer to load; a lone robot makes it much easier to understand what you are looking at.

## Moving around

Standard Omniverse camera controls:

| Action                         | Control                     |
|--------------------------------|-----------------------------|
| Orbit                          | Left-drag                   |
| Pan                            | Middle-drag                 |
| Zoom                           | Right-drag, or scroll wheel |
| Focus on an object             | Select it, press <b>F</b>   |
| Look around from where you are | Hold right mouse, move      |

**F is the one to remember.** Lost in space with nothing on screen? Click something in the Stage tree, press F, and the camera flies to it.

## The panels that matter

- **Stage** (usually top-right) — the scene tree. Every object, nested. This is the most useful panel for understanding structure: expand the robot and you see its links and joints.
- **Property** (usually bottom-right) — details of whatever is selected. Position, physics settings, joint limits.
- **Viewport** — the 3D view itself.
- **Console** — messages and errors. Worth opening when something behaves oddly.

If you close something by accident: **Window** → `\<panel name>` brings it back.

## Pressing Play

The **Play** button (▶, or the Spacebar) starts the physics simulation. **Stop** (■) resets it.

Important distinction, and a common source of confusion: - **Not playing** — you are looking at a static scene. Moving things has no consequences. - **Playing** — physics is live. Gravity applies, things fall, joints move, collisions happen.

Try this on a robot: press Play and watch. If it sags or settles slightly, that is gravity acting on the joints — the simulation is working.

**Always press Stop before editing the scene.** Editing while playing produces confusing results, and Martin's own gripper-test instructions in his README explicitly say to stop the sim first.

---

## Warnings you can safely ignore

---

These appear on a healthy load. They are noise, not problems:

- **Could not open asset @file:/C:/Users/prat615/...OpentronsDoorAdapter\_large.usdc@** A stale reference to Martin's own Windows machine, left inside the flattened USD. **The door handle is genuinely present anyway** — verified: the prim `/World/OpenTronsFlex/OpentronsDoorAdapter_large/LooRoll/Handle` exists and is valid, with its mesh and revolute joint. The geometry was baked in when the scene was flattened, so the missing external reference does not matter. A red herring worth knowing about, because it looks alarming.
  - **ConvexMeshCookingTask: failed to cook GPU-compatible mesh ... fall back to CPU** A few collision shapes are too complex for the GPU shortcut. Slower, still correct.
  - **Not all actuators are configured! ... 7 != 15** Only some joints are motorised. Intended.
  - **Unhandled attribute type VtArray<std::string>** — cosmetic material metadata.
  - **omni.isaac.dynamic\_control is deprecated** — an old API name. Harmless.
- 

## Things to try, roughly in order

---

1. **Open one robot and orbit around it.** Get comfortable with the camera. Press F on different parts.
  2. **Expand it in the Stage tree.** Find `base_link`, the arm links, the gripper fingers. This is how a robot is actually structured — a tree of links connected by joints.
  3. **Click a joint, look at the Property panel.** Find its limits and its type (revolute = rotates, prismatic = slides).
  4. **Press Play.** Watch gravity act. Press Stop.
  5. **Open the door scene** (`ENV_OT_UR_door/TrainingScene_flattened.usd`). Find the OpenTrons machine, its door, and the handle the robot is supposed to grab. This is the exact scene the RL training uses — worth seeing what the agent is working with.
  6. **Find the handle prim** at `/World/OpenTronsFlex/OpentronsDoorAdapter_large/LooRoll/Handle` in the Stage tree. Its `RevoluteJoint` is what "opening the door" physically means.
  7. **Try Martin's gripper test.** His README has a Script Editor procedure for making the gripper grasp a cube. Follow it exactly — it assumes the GUI, and it is the best way to see the gripper actually work.
  8. **Open the full lab** (`ENV_ESCBench/ESCBench.usd`) once the smaller scenes make sense.
- 

## What I could not verify

---

Everything above about *launching* is tested: the app starts, reaches "app ready" in ~16s, loads Martin's scene, finds the robot articulation, and steps physics without errors.

**The visual behaviour is not verified** — panel positions, exact menu labels, how the controls feel, whether the scene looks right. Those come from Omniverse's standard interface and may differ slightly in detail. If a menu is somewhere other than described, it is almost certainly present under a nearby menu rather than missing.

---



## Where things are

| What                       | Path                                                           |
|----------------------------|----------------------------------------------------------------|
| Isaac Sim (workstation)    | <code>/home/aidev/isaacsim/</code>                             |
| Launch script              | <code>/home/aidev/isaacsim/isaac-sim.sh</code>                 |
| Its bundled Python         | <code>/home/aidev/isaacsim/python.sh</code>                    |
| Martin's assets            | <code>/home/aidev/RL_Twin/digital_twin_models-DT_reorg/</code> |
| Asset catalogue            | <code>digital_twin_models-DT_reorg/README.md</code>            |
| The Docker/RL path instead | <code>DIGITAL_TWIN_HOWTO.md</code>                             |

## Version note

This install is **5.1.0**, deliberately. NVIDIA's download page defaults to a newer version, and Alvika specifically flagged selecting 5.1.0 — Martin's assets were authored in 5.1, and older or newer runtimes risk USD schema mismatches. The Docker container is pinned to 5.1.0 too, so both paths agree.

If you ever reinstall, check the version selector on the download page rather than taking the default.

---

## Step 3 of 3 — run the twin through Docker

RL\_DT/\_docs\_legion/DIGITAL\_TWIN\_HOWTO.md - 236 lines

*The training route. The 1-iteration smoke test is the thing to get working first. Watching it render from inside the container is the one part that was never made to work — xhost or WebRTC livestream, both untested.*

## Playing with the Digital Twin — a beginner's guide

Everything here is about the **digital twin only** (Isaac Sim + the robot scenes). Nothing here touches the RL toolkit. Written for someone new to Ubuntu and Docker.

Status as of 2026-08-28: **working**. Isaac Sim runs, the door-opening scene loads, the robot spawns, and training runs on the GPU.

---

### The mental model (read this once)

Isaac Sim is a huge program (~35 GB) that's awkward to install directly, so it lives inside a **container** — a sealed box holding the program and everything it needs. Your files stay on your computer and are "mounted" (made visible) inside that box.

Three pieces:

| Piece                 | Where it lives on your computer       | What it is                                                 |
|-----------------------|---------------------------------------|------------------------------------------------------------|
| Isaac Lab             | RL_Twin/isaacsim_dt-main/             | The container recipe + robot-learning framework (Alvika's) |
| The scenes and robots | RL_Twin/digital_twin_models-DT_reorg/ | The 3D models — robots, labs, doors (Martin's)             |
| The RL toolkit        | RL_Twin/SciOptControlToolkit/         | Not used here. Ignore for now. (Malachi's)                 |

Inside the container those appear at: - /workspace/isaacsim/ — Isaac Lab itself - /workspace/isaacsim/envs\_external/ — the scenes and robots - /workspace/isaacsim/external\_projects/ — the RL toolkit

**Important:** the container is a *running* thing, like an open application. Starting it is not the same as installing it. Installing is already done; you just start and stop it.

---

### Everyday commands

#### Is it running?

```
docker ps
```

If you see isaacsim-base listed, it's running. If nothing lists, it's stopped.

#### Start it

```
cd /home/aidev/RL_Twin/isaacsim_dt-main/docker
docker compose --env-file .env.base -f docker-compose.yaml --profile base up -d isaacsim-base
```

( -d means "run in the background and give me my terminal back")

### Get inside it

```
docker exec -it isaac-lab-base bash
```

Your prompt changes — you're now "inside" the container. Type `exit` to leave. Leaving does **not** stop it.

### Stop it

```
cd /home/aidev/RL_Twin/isaacclab_dt-main/docker
docker compose --env-file .env.base -f docker-compose.yaml down
```

---

## Run the door-opening task

This is the mobile robot (Ridgeback base + UR5e arm + gripper) learning to open an OpenTrons Flex door.

**From outside the container**, one line:

```
docker exec isaac-lab-base bash -c "cd /workspace/isaacclab/envs_external/RL_CloudTesting &&
/workspace/isaacclab/isaacclab.sh -p train_rl_games.py --task Isaac-UR5e-DoorOpen-v0 --num_envs 1 --
max_iterations 1 --headless"
```

Breaking that down: - `--task Isaac-UR5e-DoorOpen-v0` — which scenario. This is the only one set up so far. - `--num_envs 1` — how many copies of the robot to simulate at once. **1 is for testing**. Real training uses 256–4096 copies in parallel (that's the whole point of Isaac Sim — massive parallelism). Start at 1 while learning, raise it later. - `--max_iterations 1` — how many training rounds. 1 just proves it works. Real training: hundreds or thousands. - `--headless` — don't draw graphics. Faster. See below for how to actually watch it.

Expect ~40 seconds even for this tiny run — Isaac Sim takes a while to start up every time. That's normal, not a hang.

### What "it worked" looks like

Near the end you should see the robot spawn and physics start:

```
[INFO]: Time taken for scene creation : 1.19 seconds
[INFO]: Starting the simulation. This may take a few seconds. Please wait...
```

And a checkpoint file appears in `digital_twin_models-`

`DT_reorg/RL_CloudTesting/logs/rl_games/ur5e_door_open/<date>/nn/`.

### Warnings you can safely ignore

These all appear on a healthy run: - `Not all actuators are configured! ... 7 != 15` — only some robot joints are motorized. Intended. - `failed to cook GPU-compatible mesh, collision detection will fall back to CPU` — a few collision shapes are too complex for the GPU shortcut. Slower, still correct. - `torch.cuda.amp.GradScaler is deprecated` — an old-style call inside a library. Harmless. - `failed to open the default display. Can't verify X Server version` — expected in headless mode. - `IOMMU is enabled / CPU performance profile is set to powersave` — performance hints, not errors.

---

## Actually watching the robot (the fun part)

---

Headless means "no picture." To see it, drop `--headless`:

```
docker exec -it isaac-lab-base bash -c "cd /workspace/isaacclab/envs_external/RL_CloudTesting && /workspace/isaacclab/isaacclab.sh -p train_rl_games.py --task Isaac-UR5e-DoorOpen-v0 --num_envs 1 --max_iterations 10"
```

**This may not work on the first try.** Graphics from inside a container need the container to be allowed to draw on your screen, which usually means running this on your computer first:

```
xhost +local:docker
```

If the window still doesn't appear, the fallback is livestreaming instead (`--headless --livestream 2`, then connect with NVIDIA's Isaac Sim WebRTC streaming client). I have not tested either path yet — worth trying together when you have time, since seeing the robot move is the best way to build intuition.

### Replaying a trained robot

Once you've trained something worth watching:

```
docker exec -it isaac-lab-base bash -c "cd /workspace/isaacclab/envs_external/RL_CloudTesting && /workspace/isaacclab/isaacclab.sh -p play_rl_games.py --task Isaac-UR5e-DoorOpen-v0 --checkpoint logs/rl_games/ur5e_door_open/<DATE>/nn/ur5e_door_open.pth"
```

Replace `<DATE>` with the actual folder name from your training run.

---

## Things to try, roughly easiest first

1. **Re-run the 1-iteration test.** Get comfortable with the start/stop/run cycle.
2. **Turn up the numbers.** `--num_envs 64 --max_iterations 50`. Watch it get slower but learn more. Then try 256. If you run out of GPU memory, lower it — 24 GB is plenty for this task, but not infinite.
3. **Watch TensorBoard.** Training writes graphs to the `summaries/` folder. From the toolkit's environment: `tensorboard --logdir /home/aidev/RL_Twin/digital_twin_models-DT_reorg/RL_CloudTesting/logs` then open `http://localhost:6006` in a browser.
4. **Read the task definition.** `digital_twin_models-DT_reorg/RL_CloudTesting/ur5e_door_open_env_cfg.py` — this is where the scene, the robot, the rewards, and the reset behavior are all defined. It's the single most useful file for understanding how an Isaac Lab task is built.
5. **Open a scene in the Isaac Sim GUI** to look around without any RL. Requires the graphics setup from the section above.
6. **Explore the other scenes.** `envs_external/` has a tube-racking task, several lab environments, and various robots. Only the door task has RL code written for it so far.

---

## Running the RL toolkit against the digital twin

This is the newer path — Malachi's TD3 agent instead of the built-in `rl_games` PPO. Confirmed working 2026-08-28.

```
docker exec isaac-lab-base bash -c "cd /workspace/isaacclab/envs_external/RL_CloudTesting &&
/workspace/isaacclab/isaacclab.sh -p train_toolkit_td3.py --num_envs 1 --nepisodes 2 --nsteps 30 --
headless"
```

Expected output:

```
observation space : Box(-inf, inf, (43,), float32)
action space      : Box(-1.0, 1.0, (7,), float32)
agent             : TorchTD3 on cuda
Episode 0: steps=30 reward=1.308
```

**--num\_envs must be 1 here.** The adapter that connects the two deliberately handles only the single-robot case. This is fine for testing that things work, but it's slow — Isaac Sim is designed to run hundreds of robots at once, and this uses one. Making the toolkit handle many robots in parallel is the next real piece of work.

### How to inspect the environment yourself

If something doesn't line up, this prints exactly what the simulator hands back — shapes, types, which device the data is on:

```
docker exec isaac-lab-base bash -c "cd /workspace/isaacclab/envs_external/RL_CloudTesting &&
/workspace/isaacclab/isaacclab.sh -p probe_door_env.py --num_envs 1 --headless"
```

### The three files involved

- `probe_door_env.py` — the inspector above. Read-only, changes nothing.
- `isaac_gym_adapter.py` — translates between Isaac Lab's format (batched GPU tensors) and the toolkit's (plain CPU arrays). Well commented; a good file to read to understand the mismatch.
- `train_toolkit_td3.py` — the actual experiment that runs TD3 on the door task.

### Where things are

| What                                             | Path                                                                                 |
|--------------------------------------------------|--------------------------------------------------------------------------------------|
| Task code (door opening)                         | <code>digital_twin_models-DT_reorg/RL_CloudTesting/</code>                           |
| Task definition — scene, rewards, resets         | <code>.../RL_CloudTesting/ur5e_door_open_env_cfg.py</code>                           |
| Training settings (learning rate, batch size...) | <code>.../RL_CloudTesting/rl_games_ppo_cfg.yaml</code>                               |
| The 3D scene file                                | <code>digital_twin_models-DT_reorg/ENV_OT_UR_door/TrainingScene_flattened.usd</code> |
| Training output and checkpoints                  | <code>.../RL_CloudTesting/logs/rl_games/ur5e_door_open/</code>                       |
| Container config (paths, versions)               | <code>isaacclab_dt-main/docker/.env.base</code>                                      |
| Asset catalog — every robot and scene            | <code>digital_twin_models-DT_reorg/README.md</code>                                  |

## Troubleshooting

---

**"permission denied ... docker.sock"** — your user needs to be in the `docker` group. Already done on this machine, but it only takes effect after a logout or reboot.

**"nvidia-smi has failed"** (outside the container) — the GPU driver isn't loaded. This machine is pinned to kernel `6.17.0-14-generic` because that's the one with a working NVIDIA driver module. If a system update changes the boot kernel, this breaks. Check with `uname -r`.

**Changed `.env.base` and nothing happened** — mounts are fixed when the container is *created*. Stop it (`down`), then start it again to pick up changes.

**"USD file not found"** — a scene path in a config file is wrong. Paths in `ur5e_door_open_env_cfg.py` are relative to the folder you run the command from.

**Everything is slow** — the CPU is in powersave mode (there's a warning about this at startup). Fine for testing; worth changing before long training runs.

---

## One thing that's edited from the original

---

`ur5e_door_open_env_cfg.py` line 30 originally read:

```
USD_PATH = r"TrainingScene_flattened.usd"
```

It's now:

```
USD_PATH = r"../ENV_OT_UR_door/TrainingScene_flattened.usd"
```

The scene file lives in a different folder than the task scripts, and the original path assumed they were together. If you ever re-sync Martin's repo, this change may need reapplying. That line sits under a comment reading "User-editable paths," so adjusting it per-setup appears to be expected.